



AVAILABLE ON MAVEN CENTRAL

GraphCompose

Declarative document generation for Java

```
io.github.demchaav:graph-compose
```

v2.4.0

JAVA 17+

PDF

PPTX

AUTO-PAGINATION

Code › layout › finished document

Open-source Java library · MIT License



One model. PDF and PPTX outputs.

HOW IT WORKS

From one Java file to a finished document

You describe the document semantically — sections, rows, tables, charts, shapes, layers — and the engine resolves the rest. No manual coordinates, no XML templates, no per-format authoring.

1

AUTHOR

A fluent DSL describes intent, not geometry.

2

MEASURE

Every node measured twice, deterministically.

3

PAGINATE

The flow splits across pages row by row.

4

RENDER

A backend writes the bytes — PDF or PPTX.

Deterministic

The same input renders the same bytes on every machine. Layout is reproducible, not best-effort.

Regression-tested

`layoutSnapshot()` captures the resolved geometry, so a layout change fails a test instead of surfacing in a release.

One model, two formats

The same composition emits this PDF and an editable PPTX. Nothing is authored twice, so the two cannot drift.

MEASURED, NOT CLAIMED

GraphCompose against the field

The same documents through three engines — render time and peak heap, measured by this repository's own harness. Every figure on this page and the next is read from the result file at render time, not typed in.

Report size	GraphCompose	iText 9	JasperReports
1 page · 3 lines	2.0 ms · 0.2 MB	2.1 ms · 0.2 MB	4.2 ms · 0.2 MB
40 rows	4.1 ms · 1.0 MB	10.0 ms · 3.0 MB	8.3 ms · 2.5 MB
200 rows	10.9 ms · 4.0 MB	36.6 ms · 13.6 MB	13.5 ms · 10.8 MB
1000 rows	42.8 ms · 19.3 MB	189.1 ms · 67.3 MB	43.7 ms · 52.5 MB

4.4x

faster than iText 9 at 1000 rows

3.5x

lighter than iText 9

2.7x

lighter than JasperReports

Measured 2026-08-04 22:16:04 · 50 warmup / 100 measurement iterations on one machine. Read the ratios rather than the milliseconds: all three engines are measured in the same run, so their proportions survive a change of hardware while the timings do not.

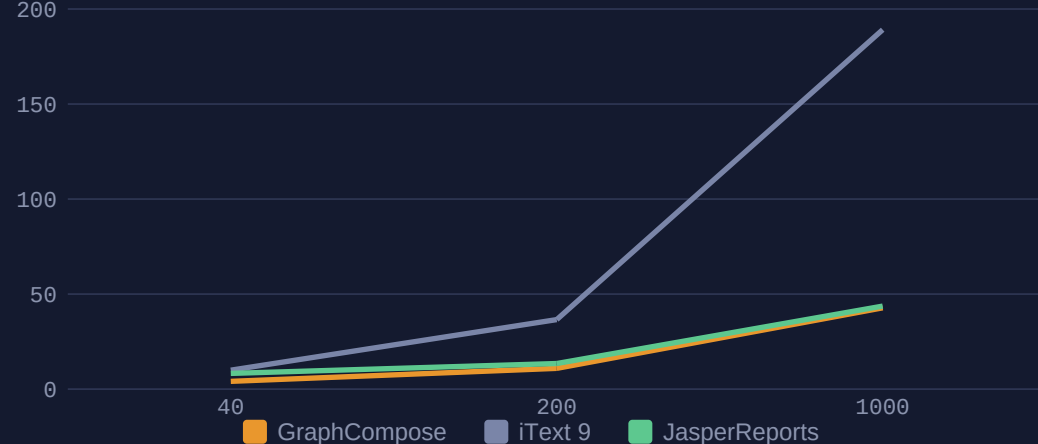
SCALING

What changes as the report grows

From 40 to 1000 rows the time lead over iText widens; JasperReports closes to roughly the same render time at the top size, while GraphCompose stays markedly lighter on memory than both throughout. Every series reads from the same file as the table on the previous page.

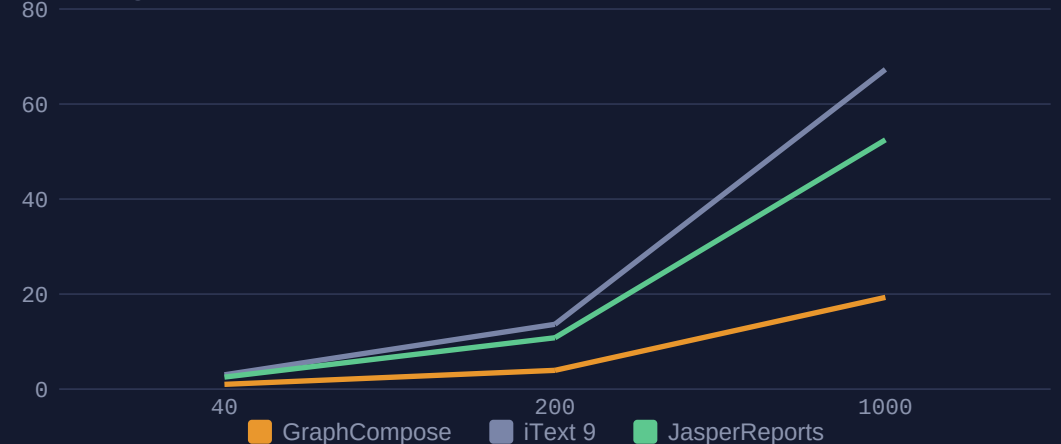
Render time vs. report size — ms

lower is faster
200



Peak heap vs. report size — MB

lower is lighter
80



Both charts are native vector output: the engine compiles them to the same shapes, lines and text frames as everything else on the page — no image is embedded, and the numbers come from the file, not the caption.

MULTILINGUAL

Documents that read right to left

A paragraph — or a table cell — says which way it runs, and Hebrew and Arabic lay out, join and mirror correctly. The PDF is painted, so the engine resolves the line itself; Word and PowerPoint order text themselves and are told what each needs instead. Five script families ship with the library.

Arabic

AMIRI

مرحبا بالعالم

Hebrew

DAVID_LIBRE

שלום עולם

Georgian

NOTO_SANS_GEORGIAN

გამარჯობა

Armenian

NOTO_SANS_ARMENIAN

Բարև աշխարհ

Korean

GOthic_A1

안녕하세요

Ordered by the algorithm

Lines resolve through UAX #9, so a Latin word or a number inside Hebrew keeps running forwards and paired punctuation faces what it encloses.

Joined, then measured

Arabic is shaped into its contextual forms before the line is measured, so a column is sized to the letters the page actually draws.

The same in three formats

PDF, PowerPoint and Word each get what their own text engine needs — the document is authored once.

Each card names its FontName constant. Every word of Hebrew and Arabic above was laid out by the engine that drew this page — no image is embedded.